

响应式布局和CSS3

2025年10月17日

目 录

01 ▶ 概述

02 ▶ 响应式布局

03 ▶ CSS3

01 概述

在网页设计的早期，页面是针对固定宽度屏幕尺寸构建的。随着终端设备的多样化发展，不同尺寸的屏幕越来越多，响应式布局的概念出现了。响应式布局是指网页布局不是固定的，而是根据不同终端设备或者不同浏览器窗口大小而呈现出不同的布局。响应式布局不仅可应用于整个页面，也可以应用于页面的一部分。

传统的开发方式是PC端开发一套页面，手机端再开发一套页面。但是这样做非常麻烦，随着不同的终端越来越多，你需要开发多个不同版本的页面。采用响应式设计时，网站仅有一个版本，能浏览全部的内容，且网站会自己重新排列以完美地适配任何尺寸的屏幕，并具有全尺寸文本，这样无需进行缩放操作。Ethan Marcotte在2010年5月份提出了响应式布局的概念，简而言之，就是一个网站能够兼容多个不同的终端设备。

02 响应式布局

响应式开发的原理

使用CSS3中的**Media Query（媒体查询）**针对不同宽度的设备设置不同的布局和样式，从而适配不同的设备。

媒体查询的语法格式如下：

```
@media media-type and (media-feature-rule) {  
    /* CSS rules go here */  
}
```

- ① 媒体类型：告诉浏览器页面显示在什么类型的设备上，常见的设备包括屏幕（screen）、打印机（print）等。
- ② 媒体特征规则：设备宽高、方向（横屏或竖屏）、分辨率等特征。
- ③ CSS规则：会在媒体特征通过且媒体类型正确的时候应用。

02 响应式布局

媒体类型

媒体类型是指设备类型，常用的有如下几种：

all：所有类型设备；

screen：计算机、平板电脑、智能手机等；

print：打印机和打印预览；

speech：语音或音频合成器；

例如，可分别设置屏幕设备和打印机的显示样式，屏幕设备上h1标题显示蓝色，所有设备包括屏幕设备或者打印机上段落文字加粗显示，代码如下：

```
<h1>响应式开发</h1>
```

```
<p>响应式开发的原理是使用CSS3中的Media Query（媒体查询）针对不同宽度的设备设置不同的布局和样式，从而适配不同的设备。
```

```
</p>
```

```
@media all{  
    p{  
        font-weight: bold;  
    }  
}
```

```
@media screen{  
    h1{  
        color:blue;  
    }  
}
```

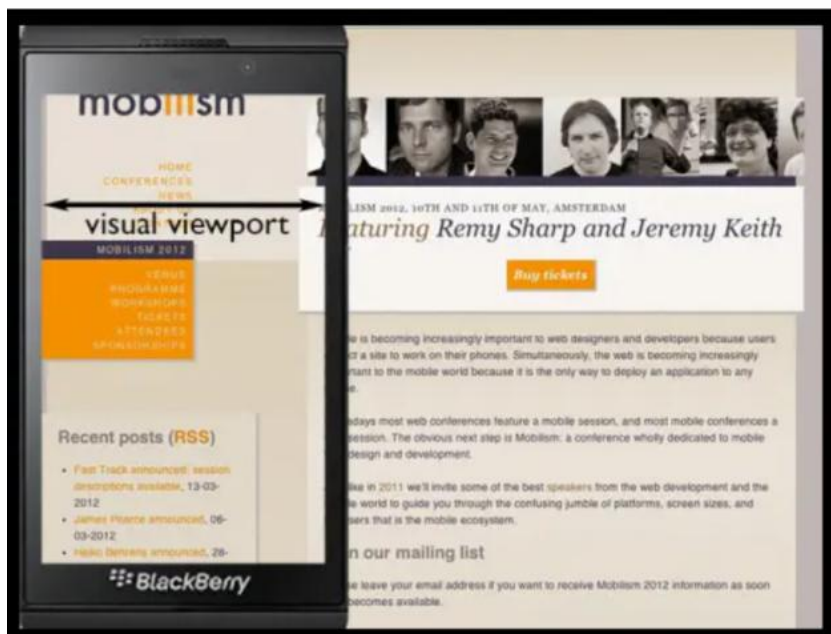
02 响应式布局

媒体特征规则

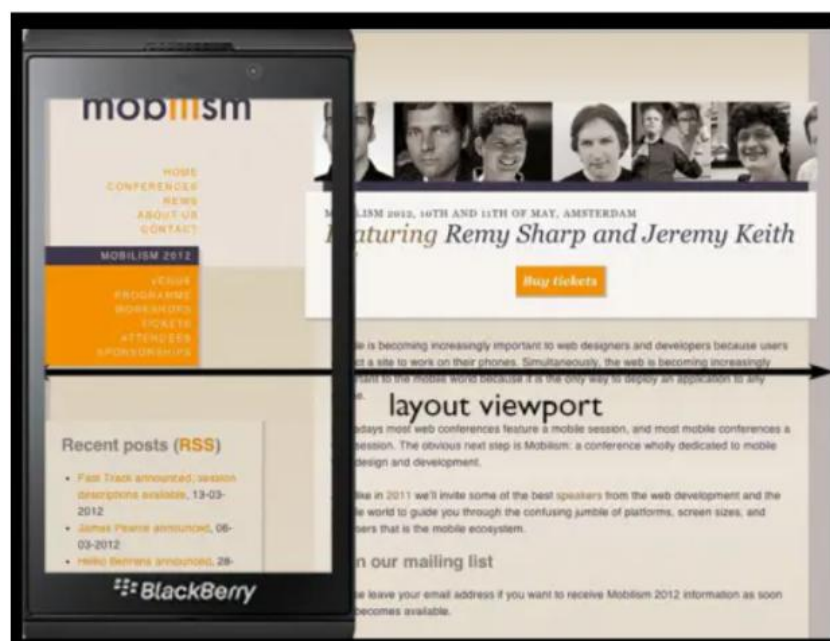
媒体特征规则就是设备宽高、横屏或竖屏、屏幕分辨率等相关特征。媒体特征涉及的一个重要概念是视口，视口是与设备相关的一个矩形区域，可以采用相对单位描述其尺寸，并规定其最大或最小宽度。

1、视口

视口分为两类：一类是设备视口，一类是布局视口。



设备本身的宽度



页面的宽度

02 响应式布局

媒体特征规则

一般最常探测的媒体特征是视口宽度，而且可以使用min-width、max-width和width媒体特征，在视口宽度大于或者小于某个大小——或者是恰好处于某个大小——的时候，应用 CSS样式规则。想在视口正好是600px的时候，让 body 的文本变为红色。

```
@media screen and (width: 600px) {  
  body {  
    color: red;  
  }  
}
```

02 响应式布局

媒体特征规则

可以应用meta元素对响应式布局视口做出规定，例如：

<meta name="viewport" content="width=device-width, initial-scale=1.0" />

- width=device-width表示布局视口宽度要与设备视口宽度缩放到一致
- initial-scale=1.0表示页面最初加载时的缩放比例，即device-width/width=1.0
- initial-scale和width可以只设置其中一个，另一个会被推算出来；如果initial-scale和width同时设置，会取计算后大的那一个，保证页面宽度刚好容纳进设备视口。

设备视口的宽度分类：（手机->平板->桌面显示器->大桌面显示器）

设备屏幕	尺寸/px
超小屏extra small	<768
小屏small	>=768
中等medium	>=992
大屏large	>=1200

02 响应式布局

复杂媒体查询

可以应用and、not等逻辑运算符或者逗号连接多个条件从而创建复杂查询。

1、and运算符

可以使用and连接多个媒体特征，表示同时满足若干条件时样式生效。

```
@media screen and (min-width: 400px) and (orientation: landscape) {  
    body {  
        color: blue;  
    }  
}
```

2、not运算符

not让整个媒体查询失效，直接反转了整个媒体查询的含义。

```
@media not all and (max-width: 900px) {  
    body {  
        color: brown;  
    }  
}
```

02 响应式布局

复杂媒体查询

3、逗号

逗号分隔多个媒体查询条件时，相当于or，表示满足其中任意一个条件都会应用对应的样式。

```
@media screen and (min-width: 400px), screen and (orientation: landscape) {  
    body {  
        color: blue;  
    }  
}
```

文本会在视口至少为 400 像素宽的时候或者设备处于横放状态的时候变为蓝色。

02 响应式布局

link方式添加媒体查询方式

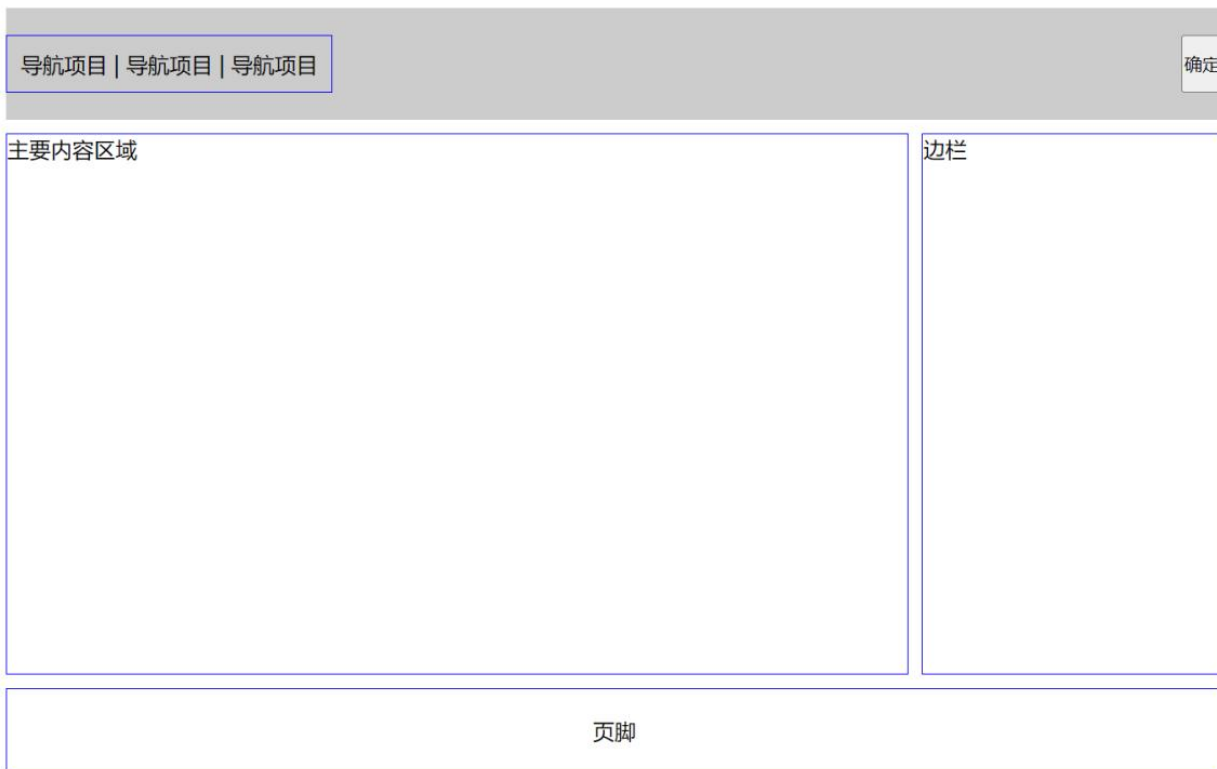
可以使用link标签的media属性完成特定样式的呈现，例如下面代码可以在宽度小于或等于800px时应用middle.css样式。

```
<link rel= “stylesheet” media= “max-width:800px” href= “middle.css” />
```

02 响应式布局

案例

1、应用媒体查询和网格布局，完成响应式布局，当宽度大于600px时，呈现如左图所示的布局，当宽度小于或等于600px时，呈现如右图所示的布局。



02 响应式布局

案例

html代码

```
<div class="container">  
  <header>  
    <nav>导航项目 | 导航项目 | 导航项目</nav>  
    <button>确定</button>  
  </header>  
  <main>主要内容区域</main>  
  <aside>边栏</aside>  
  <footer>页脚</footer>  
</div>
```

02 响应式布局

案例

style.css代码

```
*{
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
.container{
  max-width:900px;
  margin: 10px auto;
  display: grid;
  /* 总体布局分为3行2列 */
  /* 2列，宽度3:1 */
  grid-template-columns: 3fr 1fr;
  /* 网格区域命名 */
  grid-template-areas:
    "header header"
    "main aside"
    "footer footer";
  /* 网格间隙10px */
  grid-gap:10px;
}
```

```
header{
  grid-area: header;
  display: grid;
  grid-template-columns:
  1fr 1fr;
  background-color: #ccc;
  padding: 20px 0;
}
header nav{
  justify-self: start;
  border: 1px solid blue;
  padding: 10px;
}
header button{
  justify-self: end;
}
/* 主要内容区域 */
main{
  grid-area: main;
  height: 400px;
  border: 1px solid blue;
}
aside{
  grid-area:aside;
  border: 1px solid blue;
}
footer{
  grid-area: footer;
  border: 1px solid blue;
  text-align:center;
  padding: 20px 0;
}
```

```
/* 媒体查询实现响应式布局 */
/* 宽度小于或等于600px，变为单列布局 */
@media(max-width:600px){
  .container{
    grid-template-columns: 1fr;
    grid-template-areas:
      "header"
      "main"
      "aside"
      "footer";
  }
}
```

02 响应式布局

课程思政

适应变化：响应式布局强调以用户为中心的设计理念，能够适应不同的设备和屏幕尺寸，这种变化适应能力与适应社会发展和变化的能力相对应。面对在快速变化的技术环境，大家需要学会如何适应新设备和新技术。

创新和适应性：在快速变化的技术环境中，大家需要学会如何适应新设备和新技术，以及如何创造性地解决问题，以提供更好的用户体验。

终身学习能力：响应式布局的这种变化适应能力引申思考，将来步入社会同样需要具备适应社会发展和变化的能力，随着技术的不断进步，保持终身学习的习惯，使自身技能和知识不断更新

03 CSS3

1、圆角边框与阴影

圆角边框border-radius：用于添加元素的圆角边框

border-top-left-radius 左上角的形状

border-top-right-radius 右上角的形状

border-bottom-left-radius 左下角的形状

border-bottom-right-radius 右下角的形状



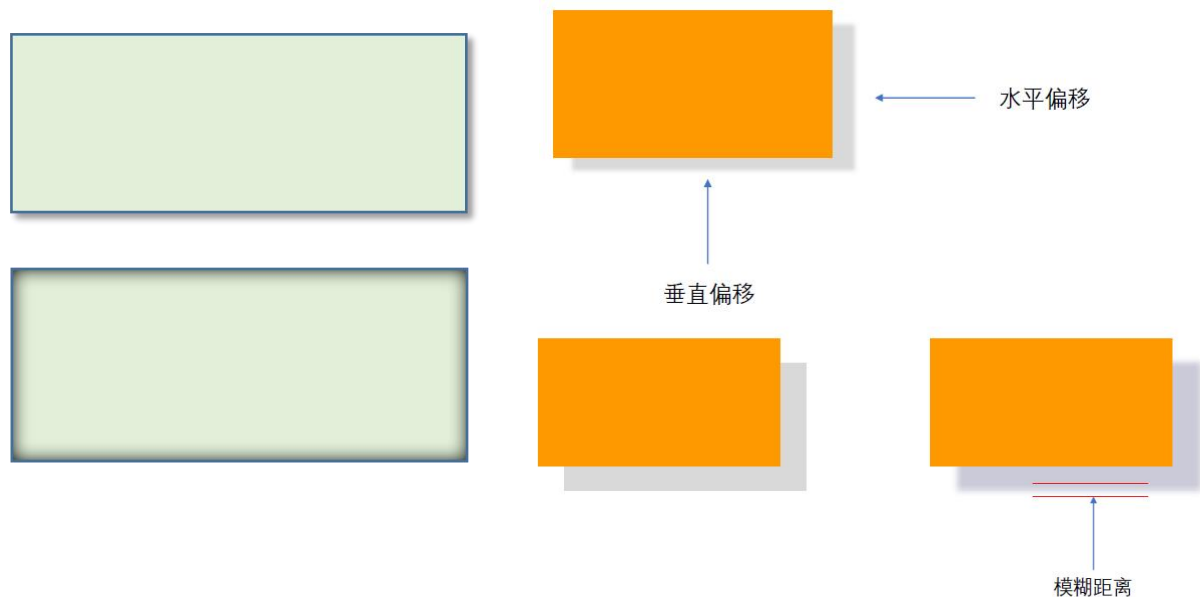
```
div {  
    height: 100px;  
    width: 150px;  
    border: 1px solid blue;  
    border-top-left-radius : 40px 20px;  
    border-bottom-right-radius: 20px;  
}
```


03 CSS3

1、圆角边框与阴影

阴影box-shadow

- 取值：inset|outset 水平偏移 垂直偏移 模糊距离 颜色
- inset可选，内部阴影，outset默认值，外部阴影
- 模糊半径决定了阴影沿偏移边界每侧的模糊范围，值越大，模糊的范围越大
- box-shadow取值可以为多个，即添加多重效果，用逗号隔开即可。



```
<div> </div>
```

```
div
```

```
{
```

```
width:300px;
```

```
height:100px;
```

```
background-color:#f90;
```

```
box-shadow: 10px 10px 5px #888;
```

```
}
```

03 CSS3

2、文本与文字

text-shadow属性：用于添加文本阴影

text-shadow: 水平偏移 垂直偏移 阴影大小 颜色;

```
h1 {  
  text-shadow: 2px 2px red;  
}
```

<h1>text-shadow</h1>

text-shadow

```
h1 {  
  text-shadow: 2px 2px 8px blueviolet;  
}
```

<h1>text-shadow</h1>

text-shadow

03 CSS3

2、文本与文字

word-wrap属性： 允许长单词、URL强制进行换行

取值： normal （默认值）

break-word

```
p.test
```

```
{
```

```
width:11em;
```

```
border:1px solid #000000;
```

```
word-wrap:break-word;
```

```
}
```

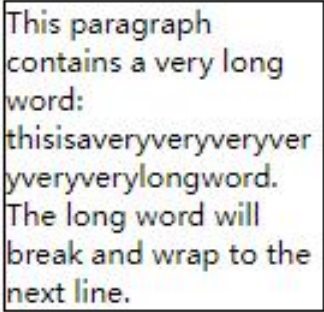
```
<p class="test">
```

```
  This paragraph contains a very long word:
```

```
  thisisaveryveryveryveryveryverylongword. The
```

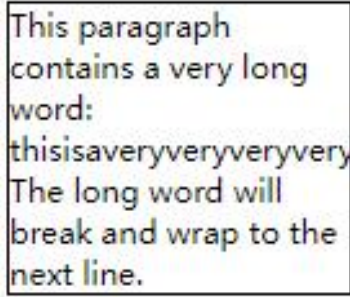
```
  long word will break and wrap to the next line.
```

```
</p>
```



This paragraph
contains a very long
word:
thisisaveryveryveryver
yveryverylongword.
The long word will
break and wrap to the
next line.

break-word



This paragraph
contains a very long
word:
thisisaveryveryveryveryveryverylongword.
The long word will
break and wrap to the
next line.

normal

3、2D变换

变换 (transform) 是CSS3中具有颠覆性的特征之一，可以实现元素的位移、旋转、缩放等效果。

变换 (transform) 可以简单理解为变形。

- 移动: translate
- 旋转: rotate
- 缩放: scale
- 倾斜: skew

3、2D变换

transform属性的常用方法

transform属性常用方法	说明
translate(x,y)	移动：沿X和Y轴
translateX(n)	移动：沿X轴
translateY(n)	移动：沿Y轴
rotate(angle)	旋转：单位是deg（角度），正数为顺时针，负数为逆时针
scale(x,y)	缩放：改变元素的宽度和高度，<1为缩小，>1为放大
scaleX(n)	缩放：改变元素的宽度
scaleY(n)	缩放：改变元素的高度
skew(x-angle,y-angle)	倾斜：沿X轴和Y轴，单位是deg（角度），一个值表示Y轴不倾斜
skewX(angle)	倾斜：沿X轴倾斜，与Y轴夹角为angle
skewY(angle)	倾斜：沿Y轴倾斜，与X轴夹角为angle

3、2D变换

移动

```
transform: translate(x, y); 或者分开写
transform: translateX(n);
transform: translateY(n);
```



旋转: transform:rotate(度数)

角度为正时，顺时针，负时，为逆时针
默认旋转的中心点是元素的中心点

transform: rotate(45deg)



变换中心点, transform-origin: x y;

- x y 默认转换的中心点是元素的中心点 (50% 50%), 相当于取值为 (center center) ;
- x y可以设置9个位置值:

水平方向:	left	center	right
	0	50%	100%
垂直方向:	top	center	bottom

3、2D变换

案例：三角形



```
.outer {  
    position: relative;  
    width: 200px;  
    height: 30px;  
    border: 1px solid #000;  
}  
.outer::after {  
    content: "";  
    position: absolute;  
    top: 10px;  
    right: 15px;  
    width: 8px;  
    height: 8px;  
    border-right: 1px solid #000;  
    border-bottom: 1px solid #000;  
    transform: rotate(45deg);  
    transition: all 0.5s;  
}  
.outer:hover::after {  
    transform: rotate(225deg);  
}
```

3、2D变换

缩放scale

`transform: scale(x,y);`

- `transform:scale(1,1)` : 宽和高都放大一倍, 相对于没有放大
- `transform:scale(2,2)` : 宽和高都放大了2倍
- `transform:scale(2)` : 只写一个参数, 第二个参数则和第一个参数一样, 相当于 `scale(2,2)`
- `transform:scale(0.5,0.5)`: 缩小

2D 变换复合写法

1. 同时使用多个转换, 其格式为`transform: translate() rotate() scale() ...`等。
2. 其顺序会影响转换的效果。(先旋转会改变坐标轴方向)
3. 当我们同时有位移和其他属性的时候, 记得要将位移放到最前, 顺序不能随意更改否则效果无法展示。

03 CSS3

3、2D变换

```
<div class="box"></div>
```

```
<style>
  .box {
    width: 100px;
    height: 100px;
    border: 1px solid blue;
    transform: translate(50px, 50px); /* 沿X轴移动50px, Y轴移动50px */
    transform: rotate(30deg); /* 顺时针旋转30度 */
    transform: scale(0.5, 2); /* 宽度缩小为原来的一半, 高度放大为原来的2倍 */
    transform: skew(10deg, 30deg); /* 沿X轴倾斜10度, 沿Y轴倾斜30度 */
  }
</style>
```

4、3D变换

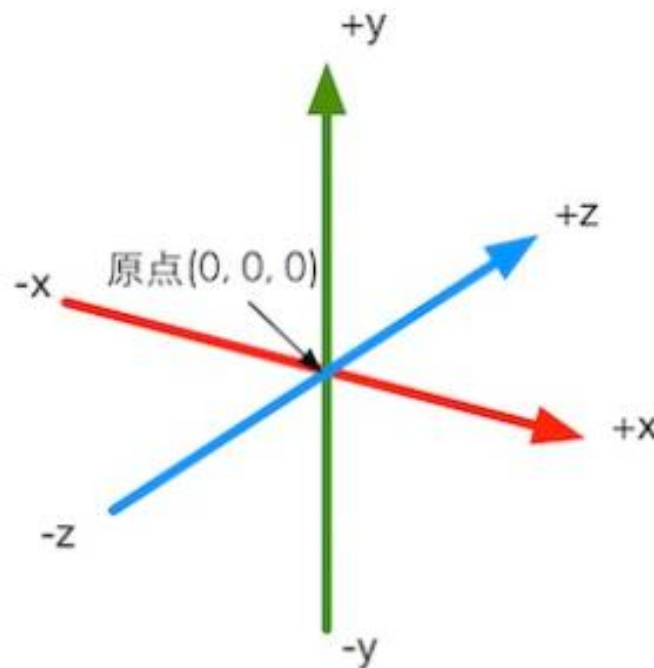
2D变换能够改变元素在X轴和Y轴方向上的特性，3D变换能改变元素在Z轴方向上的特性，并可以通过视角设置透视关系，使元素具有透视效果。

3D -> transform-style: preserve-3d;

旋转： transform属性

- transform: rotateX(deg);
- transform: rotateY(deg);
- transform: rotateZ(deg);

透视： perspective属性



03 CSS3

4、3D变换

3D变换常用属性

属性	说明
transform	2D或3D变换，取值为变换方法
transform-origin	变换的起始点，默认为元素中心点，transform-origin: x y x和y可以取值px, %, 方位名词 (left、right、top、bottom、center)
transform-style	取值为preserve-3d时，则可以产生3D变换效果
perspective	眼睛和舞台间的距离，像素为单位 值越小，透视效果越明显
perspective-origin	视线关注的原点，默认为元素中心点
backface-visibility	元素背对屏幕时是否可见，取值为hidden表示隐藏

4、3D变换

其中transform属性常用方法

旋转	移动	缩放
rotate3d(x,y,z,angle)	translate3d(x,y,z)	scale3d(x,y,z)
rotateX(angle)	translateX(x)	scaleX(x)
rotateY(angle)	translateY(y)	scaleY(y)
rotateZ(angle)	translateZ(z)	scaleZ(z)

4、3D变换

(1) 3D呈现 transform-style

- 控制子元素是否开启三维立体环境。
- transform-style: flat; 子元素不开启3d立体空间，默认的取值。
- transform-style: preserve-3d; 子元素开启立体空间。

transform-style: preserve-3d;

这段代码写给父级容器，但是它影响的是子盒子。

这个属性很重要。

4、3D变换

(2) 透视 perspective

电脑显示屏是个2D的平面，在2D平面要产生近大远小的立体视觉，但是效果只是二维的。

- 如果想要在网页上产生3D效果需要加透视（透视原理：近大远小）。
- 透视我们也称为视距：视距就是人的眼睛到屏幕的距离；
- 距离视觉点越近的在电脑平面成像越大，越远成像越小；
- 透视的单位是像素；

透视属性需要写在被观察元素的父盒子上

有了透视（perspective属性），就能看到translateZ 引起的变化了

1. translateZ: 近大远小
2. translateZ: 往外是正值
3. translateZ: 往里是负值

4、3D变换

(3) 3D移动

`translateX(x)`

`translateY(y)`

`translateZ(z)`

`translate3d(x,y,z)`

因为z轴是垂直屏幕，由里指向外面，所以默认是看不到元素在z轴的方向上移动

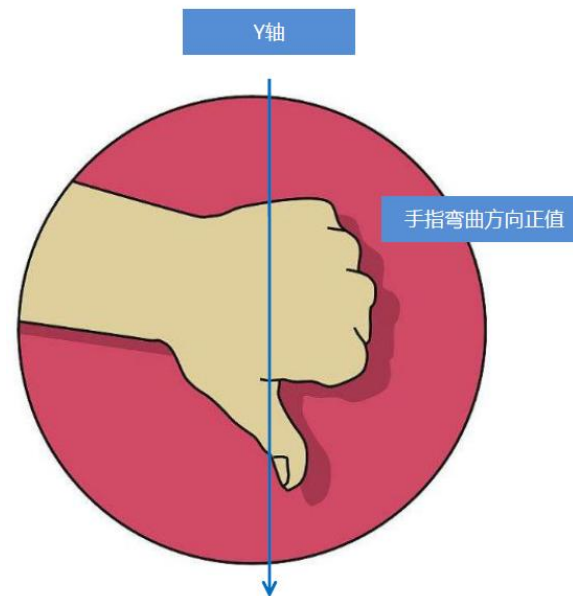
4、3D变换

(4) 3D旋转

对于元素旋转的方向的判断，需要先学习一个左手准则。

左手的大拇指指向 x轴或y轴的正方向

其余手指的弯曲方向就是该元素沿着x轴或y轴旋转的方向



03 CSS3

4、3D变换

(4) 3D旋转

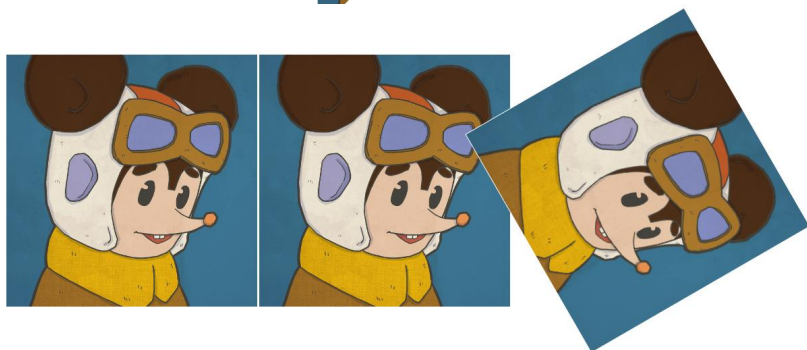
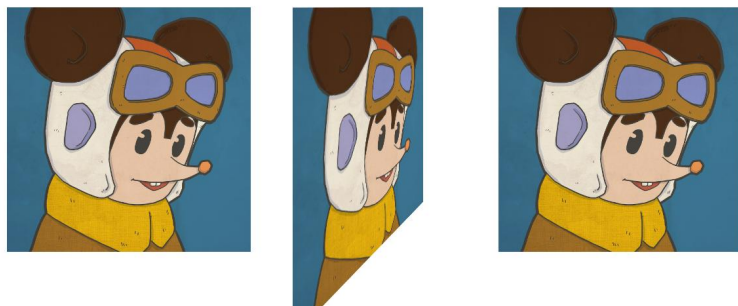
3D旋转指可以让元素在三维平面内沿着x轴，y轴，z轴或者自定义轴进行旋转。

- `transform:rotateX(45deg)`: 沿着x轴正方向旋转 45度
- `transform:rotateY(45deg)`: 沿着y轴正方向旋转 45度
- `transform:rotateZ(45deg)`: 沿着Z轴正方向旋转 45度



03 CSS3

4、3D变换



```
<body>
<div class="stage">
  <div class="box x">
    
  </div>
  <div class="box y">
    
  </div>
  <div class="box z">
    
  </div>
</div>
</body>
```

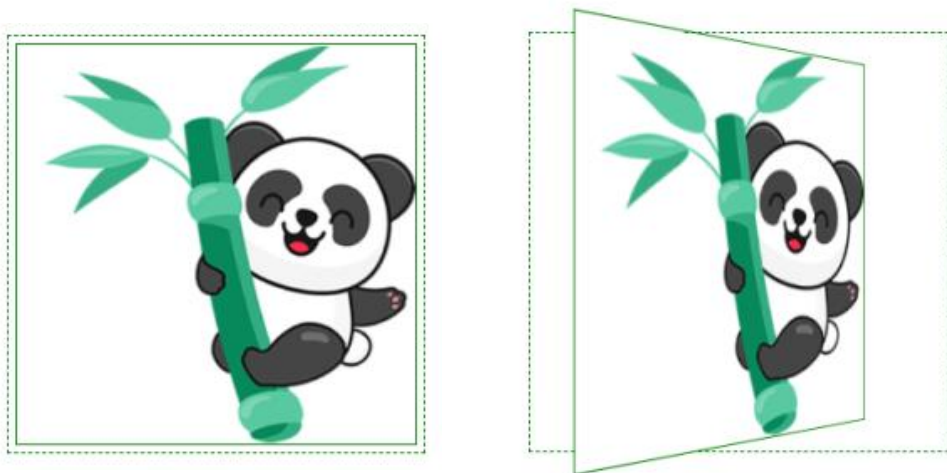
```
.stage {
  width: 900px;
  margin: 100px auto;
  perspective: 500px;
}
.box {
  width: 300px;
  height: 300px;
  float: left;
  transition: all 1s linear;
  transform-style: preserve-3d;
}
```

```
img {
  width: 100%;
  height: 100%;
  border: 1px solid #fff;
}
.x:hover {
  transform: rotateX(60deg);
}
.y:hover {
  transform: rotateY(60deg);
}
.z:hover {
  transform: rotateZ(60deg);
}
```

03 CSS3

4、3D变换

应用3D变换，当鼠标指针悬停时，完成沿Y轴旋转的效果。



```
<div class="box">  
    
</div>
```

```
.box {  
  position: relative;  
  box-sizing: border-box;  
  width: 200px;  
  height: 200px;  
  margin: 100px auto;  
  transform-style: preserve-3d;  
  border: 1px dashed green;  
  perspective: 300px;  
}  
img {  
  position: absolute;  
  width: 190px;  
  margin: 3px 3px;  
  border: 1px solid green;  
  /* 过渡效果 */  
  transition: all 0.8s;  
  /* 变换原点位于中心 */  
  /* transform-origin: 50%; */  
}  
.box:hover img {  
  /* 鼠标指针悬停，图片沿Y轴方向旋转45度 */  
  transform: rotateY(45deg);  
}
```

5、过渡

CSS3过渡可以使元素在规定时间内，从一种样式逐渐改变为另一种样式。常见的过渡属性如下：

属性	说明
transition	简写属性
transition-property	哪些CSS属性产生变化，多个属性用逗号隔开，所有属性用all表示
transition-duration	时长
transition-delay	过渡效果何时开始
transition-timing-function	过渡效果的时间曲线

transition-timing-function属性值：

- linear: 匀速
- ease: 慢-快-慢
- ease-in: 慢-快
- ease-out: 快-慢
- ease-in-out: 慢-快-慢

03 CSS3

5、过渡

例1：应用过渡效果，当鼠标指针悬停时，完成开关圆钮滑动和背景色变换效果：



```
<div id="btn">
  <i id="btn_circle"></i>
</div>
```

```
* {
  margin: 0;
  padding: 0;
}
#btn {
  position: relative; /*父层相对定位*/
  width: 48px;
  height: 20px;
  margin: 100px auto;
  border: 1px solid #ccc;
  border-radius: 20px; /*圆角边框*/
  background-color: #fff;
  transition: all 1s ease; /*1s过渡所有变化的属性，包括背景颜色*/
}
```

```
#btn_circle {
  display: inline-block;
  position: absolute; /*子层绝对定位*/
  top: 0;
  right: 28px; /*圆钮位于开关右侧28像素距离*/
  height: 18px;
  width: 18px;
  border-radius: 9px;
  border: 1px solid #ddd;
  background-color: #4cb946;
  transition: all 0.6s ease; /*0.6s过渡所有变化的属性，包括背景颜色、位置*/
}
#btn:hover {
  background-color: #4cb946; /*开关背景由白变绿*/
}
#btn:hover #btn_circle {
  right: 0; /*变换右侧距离，使圆钮有滑动效果*/
  background-color: #fff; /*圆钮背景由绿变白*/
}
```

5、过渡

例2：应用过渡效果，当鼠标指针悬停时，文字部分与父元素重合



6、动画

动画 (animation) 是CSS3中具有颠覆性的特征之一，是由静态图片连续播放而成的，静态图片称为关键帧 (Key Frame)。CSS3支持动画创建，可以代替动画图片、Flash以及部分JavaScript动态效果。相比较过渡，动画可以实现更多变化，更多控制，连续自动播放等效果。

① 用keyframes 定义动画（类似定义类选择器）

```
@keyframes 动画名称 {  
    0%{  
        width:100px;  
    }  
    100%{  
        width:200px;  
    }  
}
```

② 元素使用动画

```
div {  
    width: 200px;  
    height: 200px;  
    background-color: aqua;  
    margin: 100px auto;  
    /* 调用动画 */  
    animation-name: 动画名称;  
    /* 持续时间 */  
    animation-duration: 持续时间;  
}
```

6、动画

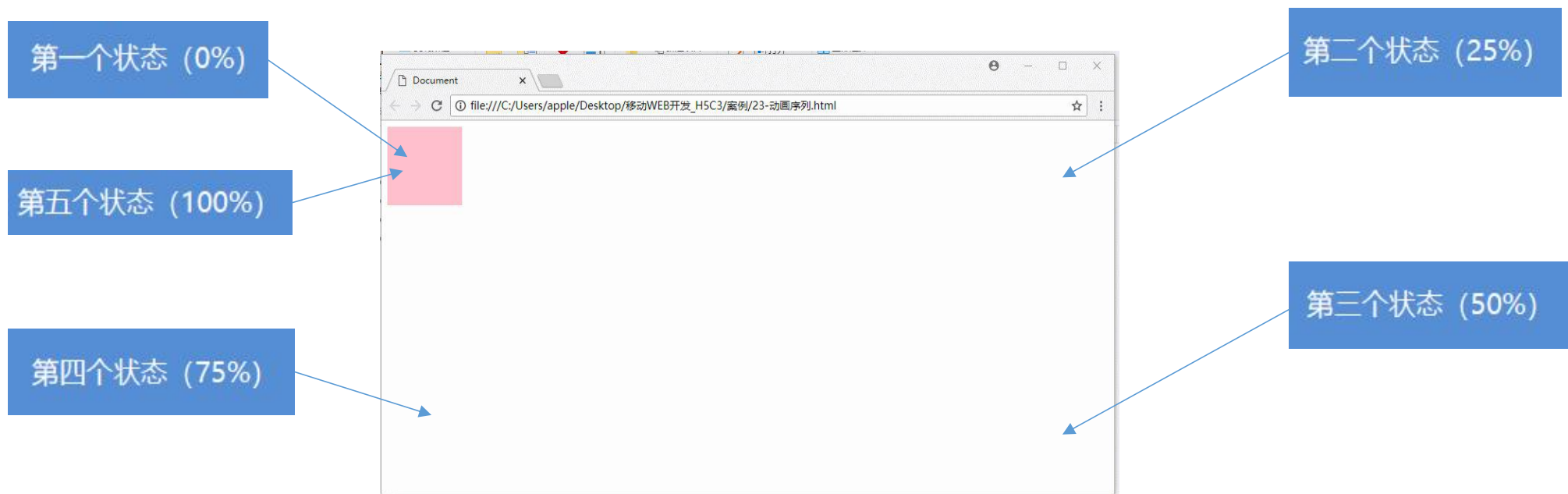
动画序列

0% 是动画的**开始**，100% 是动画的**完成**。这样的规则就是动画序列。

在 **@keyframes** 中规定某项 CSS 样式，就能创建由当前样式逐渐改为新样式的动画效果。

动画是使元素从一种样式逐渐变化为另一种样式的效果。您可以改变任意多的样式任意多的**次数**。

请用百分比来规定变化发生的时间，或用关键词 "**from**" 和 "**to**"，等同于 **0%** 和 **100%**。



6、动画

动画常用属性

属性	描述
@keyframes	规定动画。
animation	所有动画属性的简写属性，除了animation-play-state属性。
animation-name	规定@keyframes动画的名称。（必须的）
animation-duration	规定动画完成一个周期所花费的秒或毫秒，默认是0。（必须的）
animation-timing-function	规定动画的速度曲线，默认是“ease”。
animation-delay	规定动画何时开始，默认是0。
animation-iteration-count	规定动画被播放的次数，默认是1，还有infinite
animation-direction	规定动画是否在下一周期逆向播放，默认是“normal”，alternate逆播放
animation-play-state	规定动画是否正在运行或暂停。默认是“running”，还有“paused”。
animation-fill-mode	规定动画结束后状态，保持forwards回到起始backwards

6、动画

动画简写属性

animation: 动画名称 持续时间 运动曲线 何时开始 播放次数 是否反方向 动画起始或者结束的状态;

```
animation: myfirst 5s linear 2s infinite alternate;
```

- 简写属性里面不包含 animation-play-state
- 暂停动画: animation-play-state: paused; 经常和鼠标经过等其他配合使用
- 想要动画走回来，而不是直接跳回来: animation-direction : alternate
- 想要动画结束后，停在结束位置: animation-fill-mode : forwards

6、动画

速度曲线细节

animation-timing-function: 规定动画的速度曲线，默认是“ease”

值	描述
linear	动画从头到尾的速度是相同的。匀速
ease	默认。动画以低速开始，然后加快，在结束前变慢。
ease-in	动画以低速开始。
ease-out	动画以低速结束。
ease-in-out	动画以低速开始和结束。
steps()	指定了时间函数中的间隔数量（步长）

总结

利用CSS3可以实现很多图像处理软件、Flash和JavaScript才能完成的效果。

可以通过border-radius设置圆角边框；box-shadow设置阴影效果；text-shadow设置文字阴影效果。@font-face规则（前边章节网络字体部分已提及）可以实现Web字体的添加，使客户端可以不必预先安装而是从服务器端下载对应的字体文件，完成字体显示。

CSS3支持2D和3D变换，可完成缩放、移动、旋转、倾斜、透视等效果。还可以应用过渡和动画，使元素在两个或多个状态间切换。